Let's say , you have three morning chores: make toast, boil eggs, brew coffee. Each takes 2 minutes.

- **The linear way:** toast → then eggs → then coffee = **6 minutes**. You spend most of that time just waiting in line with yourself.
- **The parallel way:** start all three at once. Total time ≈ the *longest single chore* ≈ **2 minutes**.

Nothing about the chores changed. What changed is the realisation that *they don't depend on each other* — so making them wait one-behind-the-other was wasted time. That one sentence is 90% of parallelism: **if two tasks don't depend on each other, forcing them to wait in line is wasted time.**

Now map it to agents in LangGraph

In LangGraph you build your workflow as a graph — little boxes called **nodes** (each node is a step: an agent, an LLM call, a tool call), joined by **edges** (arrows that say "after this, go to that").

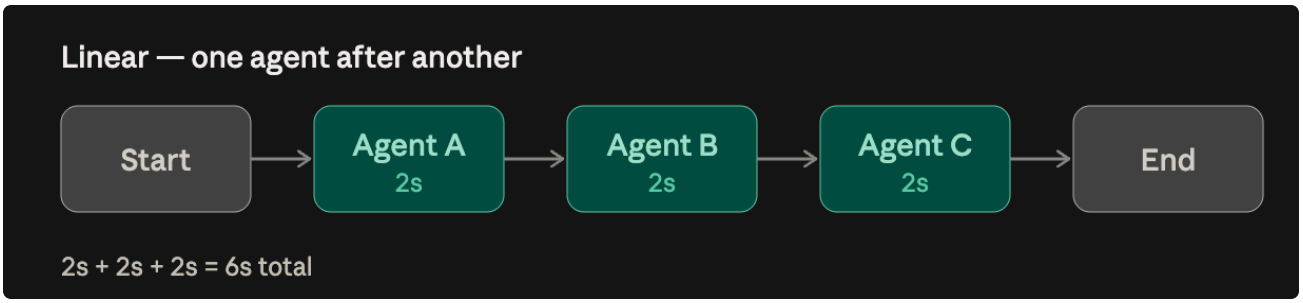
- ***Linear workflow*** = one arrow out of each node: A → B → C, one at a time. This is the default, and it's the *correct* choice whenever each step needs the previous step's output.
- ***Parallel workflow*** = one node points to several nodes at once. That split is called **fan-out**. All those nodes run together, then rejoin into a single node — that rejoin is called **fan-in**.

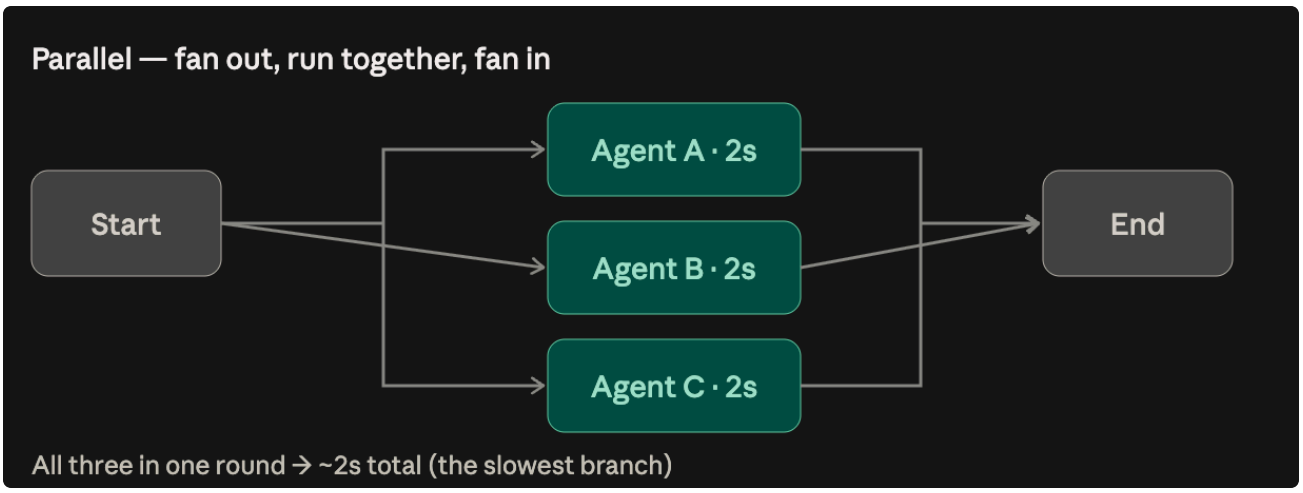
So parallelism in LangGraph is really just: fan-out (split) → run together → fan-in (join).

How it actually runs — the "rounds" idea. Under the hood LangGraph executes the graph in *rounds* (the textbook word is "super-steps," but you don't need the jargon in the room). Think of a round like a turn in a board game: *everyone who is ready to move, moves at the same time*, then the shared board updates, then the next round begins.

- Round 1: Start runs.
- Round 2: it fanned out to A, B, C → **all three run together in this single round.**
- Round 3: the fan-in node runs, now that A, B, C are all done.

The line to hammer home: **parallel nodes live in the same round; sequential nodes live in different rounds.**





When A, B and C run in parallel and all three try to write to the *same* box in the shared state, LangGraph stops and throws an error — because it doesn't know whose answer to keep.

You have to tell it *how to combine* their outputs. That instruction is called a **reducer** (for example: "append everyone's result into a list" instead of "overwrite").

I'm flagging it now because it's the number-one thing that trips people up the first time — their parallel graph errors out and they assume they broke something. They didn't; they just never told LangGraph how to merge.

Advantages, costs, and the decision

Why we reach for it (advantages)

- Speed: independent work finishes in the time of the *slowest* branch, not the *sum* of all branches.
- Natural fit for "gather from many places": search the web, query a database, read an internal doc — all at once instead of in a queue.
- "Many opinions" patterns: ask three agents the same question and combine — ensembles, voting, cross-checking.

What it costs (disadvantages)

- More moving parts: you need reducers, and debugging is harder because things don't unfold in a neat line.
- Money and rate limits: if every branch calls an LLM, you fire N calls at the same instant — a cost spike and possible throttling.
- Order is not guaranteed: you can't assume branch A finishes before branch B. (The final merged state is still predictable if your reducer is sane — but the *timing* isn't.)
- Error handling gets trickier: if one branch fails, what happens to the others and to the join?

When to use it: the branches are genuinely independent — none needs another's output — and latency or throughput matters.

When to stay linear: the steps depend on each other (you literally can't "translate the summary" before you "write the summary"); or each task is so tiny that the coordination overhead costs more than it saves.

Project 1 :

`START` fans out to three summariser nodes — `summarize_doc_1`, `summarize_doc_2`, `summarize_doc_3` — which all run in the same round. They fan in to a `combine` node that stitches the three summaries into one digest, then `END`. Split → run together → join.

All three summarisers finish around the same time and each tries to drop its summary into the *same* slot in shared state. Without instructions, LangGraph sees three writes landing on one key in one round and refuses — it doesn't know which to keep, so it errors. The fix is a one-line rule called a **reducer**: *"don't overwrite — append them into a list."* So the state is basically:

python

```
class State(TypedDict):
    documents: list[str]                    # input: the 3 docs
    summaries: Annotated[list, add]         # reducer: each branch appends
its summary
    digest: str                             # final combined output
```

That `Annotated[list, add]` is the whole trick.

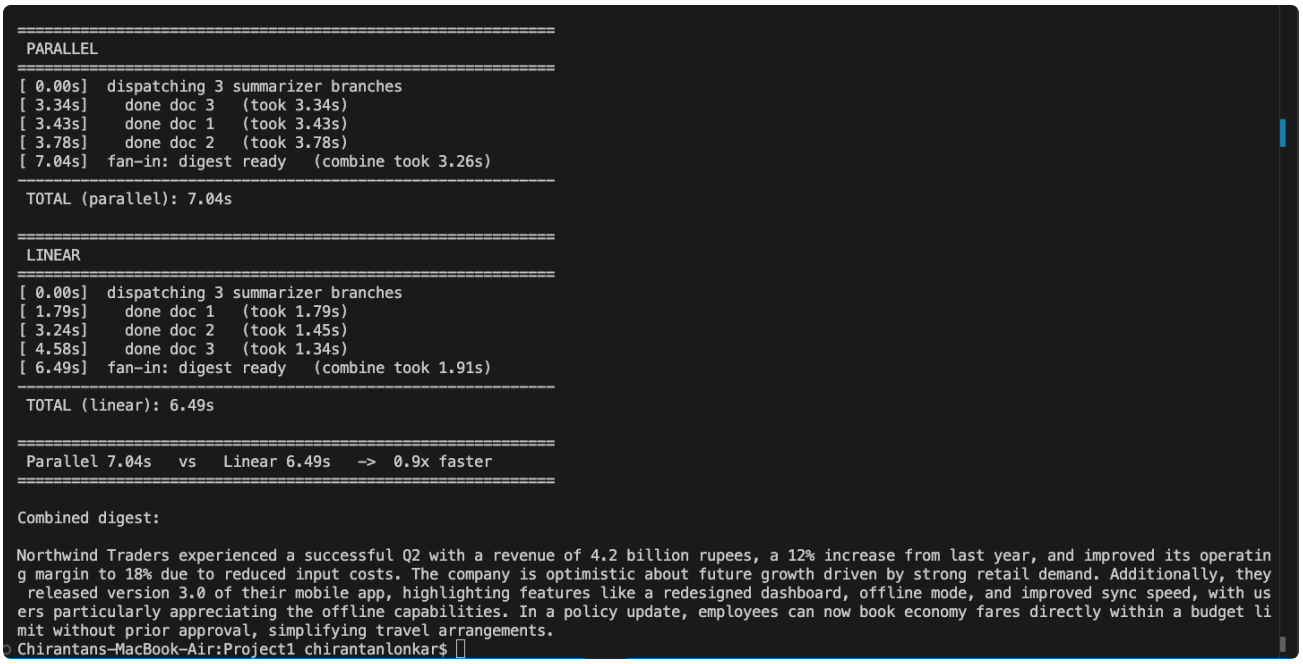
1. `sample_docs.py` — the three inputs. Point out they're unrelated, so summarizing one never needs the others. That independence is *why* parallelism is allowed here.
2. `graph.py` — the heart. Two things to slow down on:
   - The `summaries: Annotated[list, add]` line. Tell them to mentally delete `add` → the parallel run would crash, because three branches hit one box in one round and

LangGraph won't guess which to keep.
    - The two `build_*_graph` functions use the **exact same nodes** — only the edges differ. Same boxes, different arrows, completely different speed. That's the cleanest way to show that parallelism is a wiring decision, not a code-rewrite.
3. `llm.py` — deliberately boring, just the two GPT-4o calls. Keeps the credential noise out of the teaching file.
4. `run.py` — fires it parallel, then linear, prints both timelines and the speedup.

What to point at in the terminal output (three moments that *are* the lesson):

- The per-doc times are the same in both runs — parallelism didn't make any single call faster, it just overlapped them.
- Parallel total ≈ slowest branch + combine, not the sum of the three.
- `combine` takes the same time in both and can't be sped up — because it depends on all three summaries.

Output : Project 1



Parallel is not automatically faster. Look at the bottom line — parallel took 7.04s, linear took 6.49s. Parallel actually *lost* this round

The ==PARALLEL block.== All three documents were sent to GPT-4o at the same moment ("dispatching 3 summarizer branches"). Then you see them finish: doc 3 at 3.34s, doc 1 at 3.43s, doc 2 at 3.78s. Because they ran together, they all finish around the same time rather than in a neat queue. Then `fan-in: digest ready` — this is the **combine** step: a single final GPT call that takes the three separate summaries and stitches them into one paragraph (the "combined digest" printed at the very bottom). It can only start once all

three summaries exist, because it needs them as its input. `TOTAL (parallel): 7.04s` is just the stopwatch — the real time you waited from start to finish.

The ==LINEAR block.== Same four steps, but one after another. Doc 1 finishes at 1.79s, *then* doc 2 starts and finishes at 3.24s, *then* doc 3 at 4.58s, *then* combine. `TOTAL (linear): 6.49s`.

- Linear: 1.79s, 1.45s, 1.34s — about **1.5s each**
- Parallel: 3.34s, 3.43s, 3.78s — about **3.5s each**

imp : *don't parallelize when each task is small — the coordination overhead costs more than it saves.*

What is a reducer (in plain terms)

Every one of these graphs has a shared **state** — think of it as a set of labeled boxes that nodes read from and write into. One box might be `email`, another `votes`, another `decision`.

Normally, nodes take turns. Node A writes to a box, then node B writes, then node C. One write at a time — no problem.

Parallelism breaks that politeness. When five judges all run at once and all finish around the same moment, they each try to drop their vote into the **same box** (`votes`) in the **same round**. Now there's a conflict: five values, one box. Which one goes in? LangGraph refuses to guess — it stops and throws an error.

A **reducer** is simply a rule you attach to that box telling LangGraph *how to combine* several values arriving at once. The most common rule is "append them all into a list" — so instead of fighting over the box, all five votes get collected into it.

OPENAI*API_KEY="sk-proj-F3iXin8mP3sQCgR8457w104RM1ghckLS5u3EMNPzArkyd*-4QCuoYE1tqFOUpOfNjv9q8HVrxIT3BlbkFJIqpKowEtXuKyN9uotxgRVidZ6Ota2-ova5o0zKVvx3Dp70s5uLiA9RTxCEYP2TT-ISFGbfo8gA"